

REPORT ASSIGNMENT

AYA MOHAMED
SEC :1
B.NO :9

Dr.Lamiaa————— 2024

LEAF_EXAMPLE IN SIMPLE CODE

Problem Statement:

-Convert The C program of the Leaf Example to Assembly Lang. By Mars

Sample Input :

I take The input numbers from the User by Keyboard
By systemcalls

Sample Output :

for each input by the user the respond output

EXPLAIN WHY THERE ARE 2 CODES

-**First** Code is How to take input from the user by systemcalls and calc. the F value in the main and no need to call any function

-**Second** Code is also How to take input from the user by systemcalls and calc. the F value but in the leaf_example method

LEAF_EXAMPLE IN SIMPLE CODE

Problem Statement:

-Convert The C program of the Leaf Example to Assembly Lang. By Mars

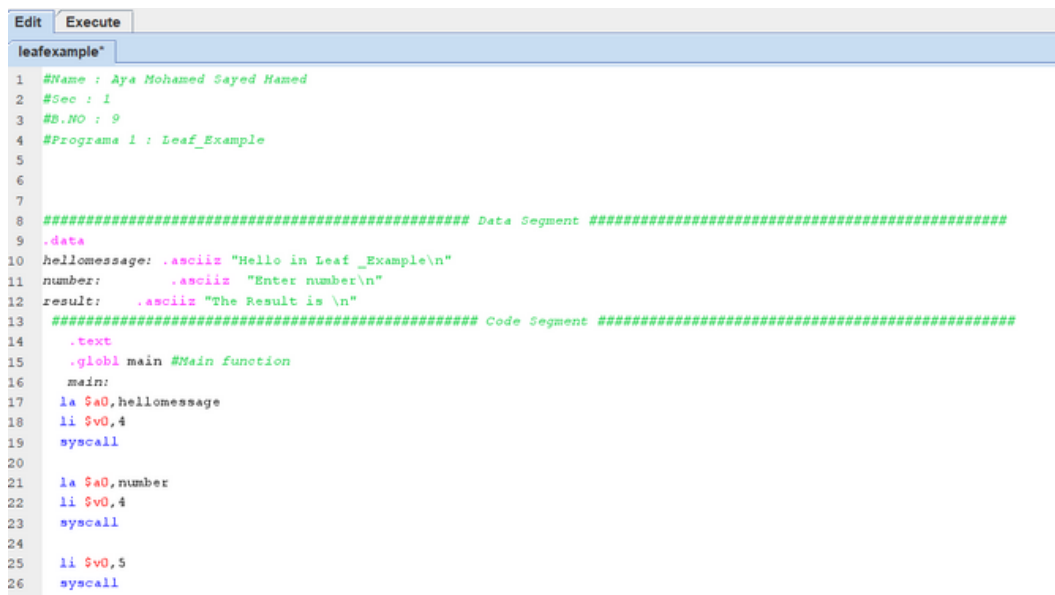
Sample Input :

I take The input numbers from the User by Keyboard
By systemcalls

Sample Output :

for each input by the user the respond output

THE CODE



```
1 #Name : Aya Mohamed Sayed Hamed
2 #Sec : 1
3 #B.NO : 9
4 #Programa 1 : Leaf_Example
5
6
7
8 ##### Data Segment #####
9 .data
10 hellomessage: .ascii "Hello in Leaf_Example\n"
11 number: .ascii "Enter number\n"
12 result: .ascii "The Result is \n"
13 ##### Code Segment #####
14 .text
15 .globl main #Main function
16 main:
17 la $a0, hellomessage
18 li $v0, 4
19 syscall
20
21 la $a0, number
22 li $v0, 4
23 syscall
24
25 li $v0, 5
26 syscall
```

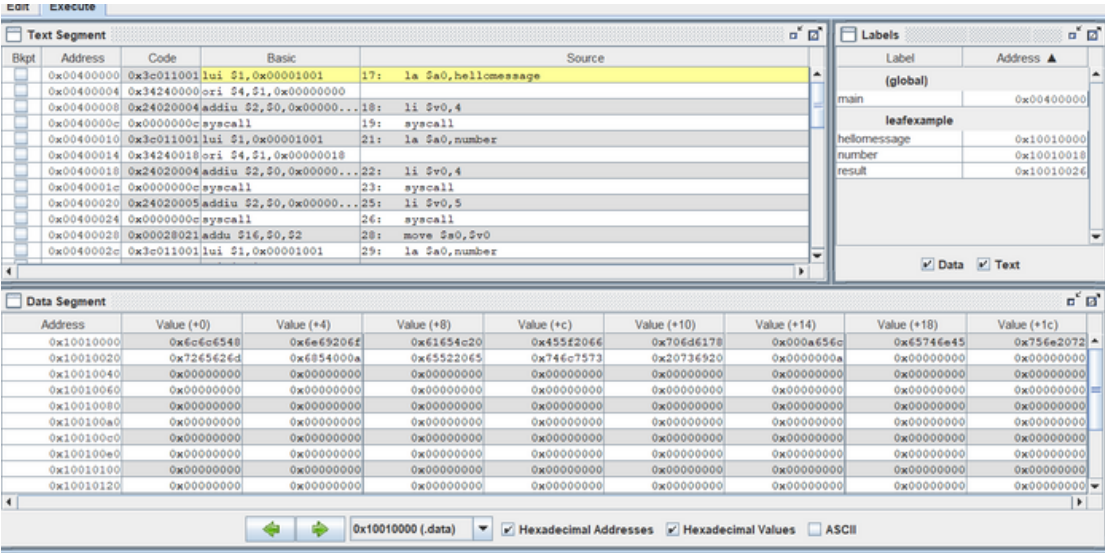
THE CODE

Edit	Execute
leafexample*	
12	li \$v0,4
13	syscall
14	
15	li \$v0,5
16	syscall
17	syscall Issue a system call : Execu
18	move \$s0,\$v0
19	la \$a0,number
20	li \$v0,4
21	
22	syscall
23	li \$v0,5
24	syscall
25	
26	move \$s1,\$v0
27	la \$a0,number
28	li \$v0,4
29	syscall
30	
31	li \$v0,5
32	syscall
33	
34	move \$s2,\$v0
35	la \$a0,number
36	li \$v0,4
37	syscall
38	
39	
40	
41	
42	
43	
44	
45	
46	
47	
48	
49	
50	
51	
52	
53	
54	
55	
56	
57	
58	
59	
60	
61	
62	
63	
64	
65	
66	
67	
68	
69	
70	
71	
72	
73	
74	
75	
76	
77	
78	
79	
80	
81	
82	
83	
84	
85	
86	
87	
88	
89	
90	
91	
92	
93	
94	
95	
96	
97	
98	
99	
100	
101	
102	
103	
104	
105	
106	
107	
108	
109	
110	
111	
112	
113	
114	
115	
116	
117	
118	
119	
120	
121	
122	
123	
124	
125	
126	
127	
128	
129	
130	
131	
132	
133	
134	
135	
136	
137	
138	
139	
140	
141	
142	
143	
144	
145	
146	
147	
148	
149	
150	
151	
152	
153	
154	
155	
156	
157	
158	
159	
160	
161	
162	
163	
164	
165	
166	
167	
168	
169	
170	
171	
172	
173	
174	
175	
176	
177	
178	
179	
180	
181	
182	
183	
184	
185	
186	
187	
188	
189	
190	
191	
192	
193	
194	
195	
196	
197	
198	
199	
200	
201	
202	
203	
204	
205	
206	
207	
208	
209	
210	
211	
212	
213	
214	
215	
216	
217	
218	
219	
220	
221	
222	
223	
224	
225	
226	
227	
228	
229	
230	
231	
232	
233	
234	
235	
236	
237	
238	
239	
240	
241	
242	
243	
244	
245	
246	
247	
248	
249	
250	
251	
252	
253	
254	
255	
256	
257	
258	
259	
260	
261	
262	
263	
264	
265	
266	
267	
268	
269	
270	
271	
272	
273	
274	
275	
276	
277	
278	
279	
280	
281	
282	
283	
284	
285	
286	
287	
288	
289	
290	
291	
292	
293	
294	
295	
296	
297	
298	
299	
300	
301	
302	
303	
304	
305	
306	
307	
308	
309	
310	
311	
312	
313	
314	
315	
316	
317	
318	
319	
320	
321	
322	
323	
324	
325	
326	
327	
328	
329	
330	
331	
332	
333	
334	
335	
336	
337	
338	
339	
340	
341	
342	
343	
344	
345	
346	
347	
348	
349	
350	
351	
352	
353	
354	
355	
356	
357	
358	
359	
360	
361	
362	
363	
364	
365	
366	
367	
368	
369	
370	
371	
372	
373	
374	
375	
376	
377	
378	
379	
380	
381	
382	
383	
384	
385	
386	
387	
388	
389	
390	
391	
392	
393	
394	
395	
396	
397	
398	
399	
400	
401	
402	
403	
404	
405	
406	
407	
408	
409	
410	
411	
412	
413	
414	
415	
416	
417	
418	
419	
420	
421	
422	
423	
424	
425	
426	
427	
428	
429	
430	
431	
432	
433	
434	
435	
436	
437	
438	
439	
440	
441	
442	
443	
444	
445	
446	
447	
448	
449	
450	
451	
452	
453	
454	
455	
456	
457	
458	
459	
460	
461	
462	
463	
464	
465	
466	
467	
468	
469	
470	
471	
472	
473	
474	
475	
476	
477	
478	
479	
480	
481	
482	
483	
484	
485	
486	
487	
488	
489	
490	
491	
492	
493	
494	
495	
496	
497	
498	
499	
500	
501	
502	
503	
504	
505	
506	
507	
508	
509	
510	
511	
512	
513	
514	
515	
516	
517	
518	
519	
520	
521	
522	
523	
524	
525	
526	
527	
528	
529	
530	
531	
532	
533	
534	
535	
536	
537	
538	
539	
540	
541	
542	
543	
544	
545	
546	
547	
548	
549	
550	
551	
552	
553	
554	
555	
556	
557	
558	
559	
560	
561	
562	
563	
564	
565	
566	
567	
568	
569	
570	
571	
572	
573	
574	
575	
576	
577	
578	
579	
580	
581	
582	
583	
584	
585	
586	
587	
588	
589	
590	
591	
592	
593	
594	
595	
596	
597	
598	
599	
600	
601	
602	
603	
604	
605	
606	
607	
608	
609	
610	
611	
612	
613	
614	
615	
616	
617	
618	
619	
620	
621	
622	
623	
624	
625	
626	
627	
628	
629	
630	
631	
632	
633	
634	
635	
636	
637	
638	
639	
640	
641	
642	
643	
644	
645	
646	
647	
648	
649	
650	
651	
652	
653	
654	
655	
656	
657	
658	
659	
660	
661	
662	
663	
664	
665	
666	
667	
668	
669	
670	
671	
672	
673	
674	
675	
676	
677	
678	
679	
680	
681	
682	
683	
684	
685	
686	
687	
688	
689	
690	
691	
692	
693	
694	
695	
696	
697	
698	
699	
700	
701	
702	
703	
704	
705	
706	
707	
708	
709	
710	
711	
712	
713	
714	
715	
716	
717	
718	
719	
720	
721	
722	
723	
724	
725	
726	
727	
728	
729	
730	
731	
732	
733	
734	
735	
736	
737	
738	
739	
740	
741	
742	
743	
744	
745	
746	
747	
748	
749	
750	
751	
752	
753	
754	
755	
756	
757	
758	
759	
760	
761	
762	
763	
764	
765	
766	
767	
768	
769	
770	
771	
772	
773	
774	
775	
776	
777	

REGISTERS VALUES

\$t1	9	0x00000003
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000004
\$s1	17	0x00000005
\$s2	18	0x00000001
\$s3	19	0x00000002
\$s4	20	0x00000006

VALUE IN MEMORY SEGMENT



SAMPLE OUTPUT & INPUT

```
Hello in Leaf _Example
Enter number
4
Enter number
5
Enter number
1
Enter number
2
The Result is
6
-- program is finished running --
```

LEAF _EXAMPLE IN SIMPLE CODE EXPLAIN THE CODE

This MIPS assembly code is a simple program called "Leaf_Example". Here's a breakdown of what it does:

1.Data Segment:

- It declares three strings: hellomessage, number, and result. These strings are used for displaying messages to the user.

2.Code Segment:

- The .text directive starts the code segment.
- main:: This label marks the beginning of the main function.
- It first prints a greeting message using the syscall with \$v0 set to 4 (print string) and the address of hellomessage loaded into \$a0.
- Then, it prints a prompt for the user to enter a number using the same syscall mechanism.
- Next, it reads an integer input from the user using syscall 5 (read integer) and stores it in register \$v0.
- It repeats the process for the next three numbers, storing them in registers \$s0 to \$s3.
- After obtaining the four numbers, it adds the first two numbers and stores the result in \$t0, adds the last two numbers and stores the result in \$t1, and then subtracts the second sum from the first to get the final result, storing it in \$s4.
- It then prints the message "The Result is " using syscall 4 (print string) with the address of result.
- The result stored in \$s4 is printed using syscall 1 (print integer).
- Finally, it exits the program by loading 10 into \$v0 (syscall code for program exit) and executing syscall 10.

LEAF_EXAMPLE IN FUNCTION CODE

Problem Statement:

-Convert The C program of the Leaf Example to Assembly Lang. By Mars

Sample Input :

I take The input numbers from the User by Keyboard
By systemcalls

Sample Output :

for each input by the user the respond output

THE CODE

```
Leafexample Function
1  #Name : Aya Mohamed Sayed Hamed
2  #Sec : 1
3  #S.NO : 9
4  #Program 1 : Leaf_Example
5
6
7
8  ##### Data Segment #####
9  .data
10 hellomessage: .ascii "Hello in Leaf_Example\n"
11 number:      .ascii "Enter number\n"
12 result:      .ascii "The Result is \n"
13 ##### Code Segment #####
14 .text
15 .globl main #Main function
16 main:
17     la $a0, hellomessage
18     li $v0, 4
19     syscall
20
21     la $a0, number
22     li $v0, 4
23     syscall
24
25     li $v0, 5
26     syscall
27
```


THE CODE

```
Leafexample Function
25  li $v0,5
26  syscall
27
28  move $s0,$v0
29  la $a0,number
30  li $v0,4
31
32  syscall
33  li $v0,5
34  syscall
35
36  move $s1,$v0
37  la $a0,number
38  li $v0,4
39  syscall
40
41  li $v0,5
42  syscall
43
44  move $s2,$v0
45  la $a0,number
46  li $v0,4
47  syscall
48
49  li $v0,5
50  syscall
51

2   move $s3,$v0
3   move $a0,$s0
4   move $a1,$s1
5   move $a2,$s2
6   move $a3,$s3
7   jal  leaf_example
8
9   la $a0,result
0   li $v0,4
1   syscall
2
3   move $a0,$s0
4   li $v0,1
5   syscall
6
7   li $v0,10
8   syscall
```

THE CODE OF FUNCTION

```
5      move $a3,$s3
7      jal   leaf_example
8
9      la $a0,result
10     li $v0,4
11     syscall
12
13     move $a0,$s0
14     li $v0,1
15     syscall
16
17     li $v0,10
18     syscall
19     leaf_example:
20     addi $sp, $sp, -4
21     sw $s0, 0($sp)
22     add $t0, $a0, $a1
23     add $t1, $a2, $a3
24     sub $s0, $t0, $t1
25     add $v0, $s0, $zero
26     lw $s0, 0($sp)
27     addi $sp, $sp, 4
28     jr $ra
29
30
```

SAMPLE OUTPUT & INPUT

```
Hello in Leaf _Example
Enter number
4
Enter number
5
Enter number
1
Enter number
2
The Result is
6
-- program is finished running --
```

REGISTERS VALUES

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000000
\$a0	4	0x00000001
\$a1	5	0x00000002
\$a2	6	0x00000001
\$a3	7	0x00000000
\$t0	8	0x00000003
\$t1	9	0x00000001
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000001
\$s1	17	0x00000002
\$s2	18	0x00000001
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7ffffeffc
\$fp	30	0x00000000
\$ra	31	0x00400094
pc		0x004000b8
hi		0x00000000
lo		0x00000000

VALUE IN MEMORY SEGMENT

Edit **Execute**

Text Segment

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x3c011001	lui \$1,0x00001001	17: la \$a0,hellmessage
☐	0x00400004	0x34240000	ori \$4,\$1,0x00000000	
☐	0x00400008	0x24020004	addiu \$2,\$0,0x000000...	18: li \$v0,4
☐	0x0040000c	0x0000000c	syscall	19: syscall
☐	0x00400010	0x3c011001	lui \$1,0x00001001	21: la \$a0,number
☐	0x00400014	0x34240018	ori \$4,\$1,0x00000018	
☐	0x00400018	0x24020004	addiu \$2,\$0,0x000000...	22: li \$v0,4
☐	0x0040001c	0x0000000c	syscall	23: syscall
☐	0x00400020	0x24020005	addiu \$2,\$0,0x000000...	25: li \$v0,5
☐	0x00400024	0x0000000c	syscall	26: syscall
☐	0x00400028	0x00002021	addu \$16,\$0,\$2	28: move \$s0,\$v0
☐	0x0040002c	0x3c011001	lui \$1,0x00001001	29: la \$a0,number

Labels

Label	Address ▲
(global)	
main	0x00400000
Leafexample Function	
leaf_example	0x004000b8
hellmessage	0x10010000
number	0x10010018
result	0x10010026

☒ Data ☒ Text

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x06cfc65d	0x06e6920f	0x061654c20	0x45522066	0x706d6178	0x000a656c	0x65746e45	0x756e2072
0x10010020	0x7265c26d	0x0654000a	0x05522065	0x746c7573	0x20736920	0x0000000a	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

LEAF_EXAMPLE IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code defines a program called "Leaf_Example" that consists of two parts: the ``main`` function and a separate function called ``leaf_example``.

Here's a breakdown:

1. ****Data Segment****:

- Defines three strings: ``hellomessage``, ``number``, and ``result``, used for displaying messages.

2. ****Code Segment****:

- ``.globl main``: Marks the beginning of the ``main`` function.
- Prints a greeting message and a prompt for the user to enter a number.
- Reads the first number input by the user and stores it in register ``$s0``.
- Repeats the process for the next three numbers, storing them in registers ``$s1`` to ``$s3``.
- Then, it moves the contents of registers ``$s0`` to ``$s3`` into registers ``$a0`` to ``$a3``, respectively, which are argument registers for function calls in MIPS.
- Calls the ``leaf_example`` function using the ``jal`` (jump and link) instruction.
- After returning from the function call, it prints the message "The Result is" and then prints the content of register ``$s0``, which holds the result calculated in the ``leaf_example`` function.
- Finally, it exits the program.

LEAF_EXAMPLE IN FUNCTION CODE EXPLAIN THE CODE

3. ****`leaf\_example` function****:

- Begins with setting up the stack frame by decrementing the stack pointer (`\$sp`) and saving the value of register `\$s0` onto the stack.
- Calculates the result by adding the first two arguments (`\$a0` and `\$a1`) and storing the sum in `\$t0`, adding the last two arguments (`\$a2` and `\$a3`) and storing the sum in `\$t1`, and then subtracting `\$t1` from `\$t0` and storing the result in `\$s0`.
- Copies the result to register `\$v0`, which is used to return values from functions in MIPS.
- Restores the value of `\$s0` from the stack.
- Adjusts the stack pointer back to its original position.
- Jumps back to the calling function using the `\$ra` register (return address).

F TO C TEMP. CONVERTER IN SIMPLE CODE

Problem Statement:

-Convert The C program of the F to C temp. Example to Assembly Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord
By systemcalls

Sample Output :

for each input by the user the respond output

EXPLAIN WHY THERE ARE 2 CODES

-**First** Code is How to take input from the user by systemcalls and calc. the C temp. value in the main and no need to call any function

-**Second** Code is also How to take input from the user by systemcalls and calc. the C value but in the leaf_example method

F TO C TEMP. CONVERTER IN SIMPLE CODE

Problem Statement:

-Convert The C program of the F to C temp. Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord
By systemcalls

Sample Output :

for each input by the user the respond output

CODE

```
#Name : Aya Mohamed Sayed Hamed
#Sec : 1
#B.NO : 9
#Programa 1 : F To C Converter

##### Data Segment #####
.data
hellomessage: .ascii "Hello in F To C Temp _Example\n"
number: .ascii "Enter number\n"
result: .ascii "The Result is \n"
delta: .float 32.0
scalar: .float 5.0
another_scalar: .float 9.0

##### Code Segment #####
.text
.globl main #Main function
main:
la $a0, hellomessage
li $v0, 4
syscall

la $a0, number
li $v0, 4
syscall
```


THE CODE

```
li $v0,6
syscall

# Convert Fahrenheit to Celsius
l.s $f1, delta
l.s $f2, scalar
l.s $f3, another_scalar

sub.s $f0, $f0, $f1
mul.s $f0, $f0, $f2
div.s $f0, $f0, $f3

la $a0,result
li $v0,4
syscall
li $v0, 2
mov.s $f12, $f0 # Move contents of register $f0 to register $f12
syscall

li $v0,10
syscall
```

SAMPLE OUTPUT & INPUT

```
Hello in F To C Temp _Example
Enter number
75
The Result is
23.88889
-- program is finished running --
```

REGISTERS VALUES

\$gp	28	0x10008000
\$sp	29	0x7ffffeffc
\$fp	30	0x00000000
\$ra	31	0x00000000
pc		0x00400070
hi		0x00000000
lo		0x00000000

\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000000
\$a0	4	0x1001002d

VALUE IN MEMORY SEGMENT

Text Segment				
Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x3c011001	lui \$1,0x00001001	20: la \$a0,hellomessage
☐	0x00400004	0x34240000	ori \$4,\$1,0x00000000	
☐	0x00400008	0x24020004	addiu \$2,\$0,0x0000...21:	li \$v0,4
☐	0x0040000c	0x0000000c	syscall	22: syscall
☐	0x00400010	0x3c011001	lui \$1,0x00001001	24: la \$a0,number
☐	0x00400014	0x3424001f	ori \$4,\$1,0x0000001f	
☐	0x00400018	0x24020004	addiu \$2,\$0,0x0000...25:	li \$v0,4
☐	0x0040001c	0x0000000c	syscall	26: syscall
☐	0x00400020	0x24020006	addiu \$2,\$0,0x0000...28:	li \$v0,6
☐	0x00400024	0x0000000c	syscall	29: syscall
☐	0x00400028	0x3c011001	lui \$1,0x00001001	33: l.s \$f1,delta
☐	0x0040002c	0xc4210040	lwc1 \$f1,0x00000040...	

Labels	
Label	Address ▲
(global)	
main	0x00400000
F to C Converter simp...	
hellomessage	0x10010000
number	0x1001001f
result	0x1001002d
delta	0x10010040
scalar	0x10010044
another_scalar	0x10010048

☒ Data ☒ Text

F TO C TEMP. CONVERTER IN SIMPLE CODE

EXPLAIN THE CODE

This MIPS assembly code is a program that converts Fahrenheit temperature to Celsius. Here's a breakdown of its functionality:

1. ****Data Segment****:

- Declares four strings: ``hellomessage``, ``number``, ``result``, used for displaying messages to the user.
- Defines two floating-point constants: ``delta`` with a value of 32.0 and ``scalar`` with a value of 5.0, and ``another_scalar`` with a value of 9.0. These constants are used in the conversion formula.

2. ****Code Segment****:

- ``.globl main``: Marks the beginning of the ``main`` function.
- Prints a greeting message and a prompt for the user to enter a number (Fahrenheit temperature).
- Reads the Fahrenheit temperature input by the user using `syscall 6` (read float).
- Converts the Fahrenheit temperature to Celsius using the formula: ``Celsius = (Fahrenheit - 32) * 5 / 9``.
- Prints the message "The Result is" using `syscall 4` (print string) with the address of ``result``.
- Prints the result (Celsius temperature) using `syscall 2` (print float).
- Exits the program.

The code achieves temperature conversion by performing floating-point arithmetic operations on the input Fahrenheit temperature (``$f0``) using the defined constants (``delta``, ``scalar``, ``another_scalar``) and MIPS floating-point instructions (``l.s``, ``sub.s``, ``mul.s``, ``div.s``). Finally, it displays the result (Celsius temperature) to the user.

F TO C TEMP. CONVERTER IN FUNCTION CODE

Problem Statement:

-Convert The C program of the F to C temp. Example to Assembly Lang. By Mars

Sample Input :

I take The input numbers from the User by Keyboard
By systemcalls

Sample Output :

for each input by the user the respond output

CODE

```
##### Data Segment #####  
.data  
hellomessage: .asciiz "Hello in F To C Temp _Example\n"  
number:       .asciiz "Enter number\n"  
result:       .asciiz "The Result is \n"  
delta: .float 32.0  
scalar: .float 5.0  
another_scalar: .float 9.0  
  
##### Code Segment #####  
.text  
.globl main #Main function  
  
main:  
    la $a0, hellomessage  
    li $v0, 4  
    syscall  
  
    la $a0, number  
    li $v0, 4  
    syscall  
  
    li $v0, 6  
    syscall  
  
    .text Subsequent items (instructions) stored in Text segment at next available address
```

THE CODE AND FUNCTION CODE

```
# Save return address
jal convertFtoC
mov.s $f12, $f0      # Move Fahrenheit value to argument register for function call
# Print the result
la $a0, result
li $v0, 4
syscall
li $v0, 2
syscall

li $v0, 10
syscall
# Function to convert Fahrenheit to Celsius
convertFtoC:

l.s $f1, delta        # Load 32.0 into $f1
l.s $f2, scalar        # Load 5.0 into $f2
l.s $f3, another_scalar # Load 9.0 into $f3

sub.s $f0, $f0, $f1    # Fahrenheit - 32
mul.s $f0, $f0, $f2    # (Fahrenheit - 32) * 5
div.s $f0, $f0, $f3    # (Fahrenheit - 32) * 5 / 9

jr $ra                # Return to caller
```

SAMPLE OUTPUT & INPUT

```
Hello in F To C Temp _Example
Enter number
1
The Result is
-17.222221
```

REGISTERS VALUES

\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000000
\$a0	4	0x1001002d
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000

\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7ffffeffc
\$fp	30	0x00000000
\$ra	31	0x0040002c
pc		0x00400050
hi		0x00000000
lo		0x00000000

VALUE IN MEMORY SEGMENT

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x3e011001	lui \$1,0x00001001	16: la \$a0,helloMessage
	0x00400004	0x34240000	ori \$4,\$1,0x00000000	
	0x00400008	0x24020004	addiu \$2,\$0,0x00000...	17: li \$v0,4
	0x0040000c	0x0000000c	syscall	18: syscall
	0x00400010	0x3e011001	lui \$1,0x00001001	20: la \$a0,number
	0x00400014	0x3424001f	ori \$4,\$1,0x0000001f	
	0x00400018	0x24020004	addiu \$2,\$0,0x00000...	21: li \$v0,4
	0x0040001c	0x0000000c	syscall	22: syscall
	0x00400020	0x24020006	addiu \$2,\$0,0x00000...	24: li \$v0,6
	0x00400024	0x0000000c	syscall	25: syscall
	0x00400028	0x0c100014	jal 0x00400050	29: jal convertFtoC
	0x0040002c	0x46000306	mov.s \$f12,\$f0	30: mov.s \$f12, \$f0 # Move Fahrenheit value to arg...

Labels

Label	Address
(global)	
main	0x00400000
Function F To C	
convertFtoC	0x00400050
helloMessage	0x10010000
number	0x1001001f
result	0x1001002d
delta	0x10010040
scalar	0x10010044
another_scalar	0x10010048

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6c6c6548	0x6e69206f	0x54204620	0x2043206f	0x706d6554	0x78455f20	0x6c706d61	0x45000a65
0x10010020	0x7265746e	0x6d756e20	0x0a726562	0x65685400	0x73655220	0x20746c75	0x0a207369	0x00000000
0x10010040	0x42000000	0x40a00000	0x41100000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (.data)

☒ Hexadecimal Addresses
 ☒ Hexadecimal Values
 ☐ ASCII

F TO C TEMP. CONVERTER IN FUNCTION CODE

EXPLAIN THE CODE

This MIPS assembly code is a program that converts Fahrenheit temperature to Celsius using a function. Here's the breakdown of its functionality:

1. ****Data Segment****:

- Declares four strings: ``hellomessage``, ``number``, ``result``, used for displaying messages to the user.
- Defines three floating-point constants: ``delta`` with a value of 32.0, ``scalar`` with a value of 5.0, and ``another_scalar`` with a value of 9.0. These constants are used in the conversion formula.

2. ****Code Segment****:

- ``.globl main``: Marks the beginning of the ``main`` function.
- Prints a greeting message and a prompt for the user to enter a number (Fahrenheit temperature).
- Reads the Fahrenheit temperature input by the user using syscall 6 (read float).
- Calls the function ``convertFtoC`` to perform the Fahrenheit to Celsius conversion.
- Moves the result (Celsius temperature) from the return value register ``$f0`` to ``$f12``, the argument register for syscall 2 (print float).
- Prints the message "The Result is" using syscall 4 (print string) with the address of ``result``.
- Prints the Celsius temperature using syscall 2 (print float).
- Exits the program.

3. ****`convertFtoC` function****:

- This function is called from the ``main`` function.
- Loads the floating-point constants ``delta``, ``scalar``, and ``another_scalar`` from memory.
- Calculates the Celsius temperature using the formula: ``Celsius = (Fahrenheit - 32) * 5 / 9``.
- Returns the result by jumping back to the caller using the ``$ra`` register.

COPY STRING IN FUNCTION CODE

Problem Statement:

-Convert The C program of the copy string. Example to Assembly Lang. By Mars

Sample Input :

I take The input numbers from the User by Keyboard
By systemcalls

Sample Output :

for each input by the user the respond output

CODE

Learexample Function	F to C Converter simpler code without FUNCTION	Function F to C	String copy
<pre>1 #Name : Aya Mohamed Sayed Hamed 2 #Sec : 1 3 #B.NO : 9 4 #Programa 1 : String copy_Example 5 6 7 8 ##### Data Segment ##### 9 .data 10 hellomessage: .ascii "Hello in String copy _Example\n" 11 string: .ascii "Enter String \n" 12 result: .ascii "The Result is \n" 13 buffer1: .space 256 # Allocate space for the input string 14 buffer2: .space 256 # Allocate space for the input string 15 ##### Code Segment ##### 16 .text 17 .globl main #Main function 18 main: 19 la \$a0, hellomessage 20 li \$v0, 4 21 syscall 22 23 la \$a0, string 24 li \$v0, 4 25 syscall 26 27 li \$v0, 8</pre>			

THE CODE

```
28     la $a0, buffer1      # load address of buffer
29     li $a1, 256          # maximum number of characters to read
30     syscall
31     la $a1, buffer2
32 jal  strcpy_function
33
34 move $a0, $a1
35
36
37     # Print the string
38     li $v0, 4             # syscall code for print_string
39     la $a0, buffer1      # load address of buffer
40     syscall
41
42     # Exit program
43     li $v0, 10           # syscall code for exit
44     syscall
45 strcpy_function:
46
47 addi $sp, $sp, -4 # adjust stack for 1 item
48 sw $s0, 0($sp) # save $s0
49 add $s0, $zero, $zero # i = 0
50 L1: add $t1, $s0, $a0 # addr of y[i] in $t1
51 lbu $t2, 0($t1) # $t2 = y[i]
52 add $t3, $s0, $a1 # addr of x[i] in $t3
53 sb $t2, 0($t3) # x[i] = y[i]
54 beq $t2, $zero, L2 # exit loop if y[i] == 0

strcpy_function:

addi $sp, $sp, -4 # adjust stack for 1 item
sw $s0, 0($sp) # save $s0
add $s0, $zero, $zero # i = 0
L1: add $t1, $s0, $a0 # addr of y[i] in $t1
lbu $t2, 0($t1) # $t2 = y[i]
add $t3, $s0, $a1 # addr of x[i] in $t3
sb $t2, 0($t3) # x[i] = y[i]
beq $t2, $zero, L2 # exit loop if y[i] == 0
addi $s0, $s0, 1 # i = i + 1
j L1 # next iteration of loop
L2: lw $s0, 0($sp) # restore saved $s0
addi $sp, $sp, 4 # pop 1 item from stack
jr $ra # and return
```

REGISTERS VALUES

Registers	Coproc 1	Coproc 0	
Name	Number		Value
\$zero	0		0x00000000
\$at	1		0x10010000
\$v0	2		0x0000000a
\$v1	3		0x00000000
\$a0	4		0x1001003e
\$a1	5		0x1001013e
\$a2	6		0x00000000
\$a3	7		0x00000000
\$t0	8		0x00000000
\$t1	9		0x10010043
\$t2	10		0x00000000
\$t3	11		0x10010143
\$t4	12		0x00000000
\$t5	13		0x00000000
\$t6	14		0x00000000
\$t7	15		0x00000000
\$s0	16		0x00000000
\$s1	17		0x00000000
\$s2	18		0x00000000
\$s3	19		0x00000000
\$s4	20		0x00000000
\$s5	21		0x00000000
\$s6	22		0x00000000
\$s7	23		0x00000000
\$t8	24		0x00000000
\$t9	saved temporary (preserved across call)		0x00000000
\$k0	26		0x00000000
\$k1	27		0x00000000
\$gp	28		0x10008000
\$sp	29		0x7fffeffc
\$fp	30		0x00000000
\$ra	31		0x00400040
pc			0x0040005c
hi			0x00000000
lo			0x00000000

SAMPLE OUTPUT & INPUT

```
Hello in String copy _Example
Enter String
play
play
```

VALUE IN MEMORY SEGMENT

EditExecute

Text Segment

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x3c011001	lui \$1,0x00001001	19: la \$a0,hellomessage
☐	0x00400004	0x34240000	ori \$4,\$1,0x00000000	
☐	0x00400008	0x24020004	addiu \$2,\$0,0x00000...	20: li \$v0,4
☐	0x0040000c	0x0000000c	syscall	21: syscall
☐	0x00400010	0x3c011001	lui \$1,0x00001001	23: la \$a0,string
☐	0x00400014	0x3424001f	ori \$4,\$1,0x0000001f	
☐	0x00400018	0x24020004	addiu \$2,\$0,0x00000...	24: li \$v0,4
☐	0x0040001c	0x0000000c	syscall	25: syscall
☐	0x00400020	0x24020008	addiu \$2,\$0,0x00000...	27: li \$v0,8
☐	0x00400024	0x3c011001	lui \$1,0x00001001	28: la \$a0,buffer1 # load address of buffer
☐	0x00400028	0x3424003e	ori \$4,\$1,0x0000003e	
☐	0x0040002c	0x24050100	addiu \$5,\$0,0x00000...	29: li \$a1, 256 # maximum number of characters t...

Labels

Label	Address
(global)	
main	0x00400000
String copy	
stringcopy_function	0x0040005c
L1	0x00400068
L2	0x00400084
hellomessage	0x10010000
string	0x1001001f
result	0x1001002e
buffer1	0x1001003e
buffer2	0x1001013e

☒ Data☒ Text

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6c6c6548	0x6e69206f	0x72745320	0x20676e69	0x79706f63	0x78455f20	0x6c706d61	0x45000a65
0x10010020	0x7265746e	0x72745320	0x20676e69	0x6854000a	0x65522065	0x746c7573	0x20736920	0x6c70000a
0x10010040	0x000a7961	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x6c700000

0x10010000 (.data)

☒ Hexadecimal Addresses☒ Hexadecimal Values☐ ASCII

COPY STRING IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code is a program that copies a string from one buffer to another buffer. Here's the breakdown of its functionality:

1. ****Data Segment****:

- Declares three strings: ``hellomessage``, ``string``, and ``result``, used for displaying messages to the user.
- Defines two buffers: ``buffer1`` and ``buffer2``, each allocated with space to hold 256 characters. These buffers are used for storing the input string and the copied string.

2. ****Code Segment****:

- ``.globl main``: Marks the beginning of the ``main`` function.
- Prints a greeting message and a prompt for the user to enter a string.
- Reads a string input by the user using `syscall 8` (read string) into ``buffer1``.
- Calls the function ``stringcopy_function`` to copy the string from ``buffer1`` to ``buffer2``.
- Prints the copied string stored in ``buffer1``.
- Exits the program.

3. ****`stringcopy\_function` function****:

- This function is called from the ``main`` function.
- Copies the string from ``buffer1`` to ``buffer2``.
- It uses a loop to iterate through each character of the string stored in ``buffer1``.
- For each character, it loads it into a temporary register ``$t2``, then stores it in the corresponding position in ``buffer2``.
- The loop continues until it encounters a null terminator (end of string).
- After copying, it returns to the caller.

FACTORIAL IN FUNCTION CODE

Problem Statement:

-Convert The C program of the Factorial. Example to Assembly Lang. By Mars

Sample Input :

I take The input numbers from the User by Keyboard
By systemcalls

Sample Output :

for each input by the user the respond output

CODE

```
#Name : Aya Mohamed Sayed Hamed
#Sec : 1
#B.NO : 9
#Programa 1 : Factorial_Example

##### Data Segment #####
.data
hellomessage: .ascii "Hello in Factorial _Example\n"
number:       .ascii "Enter number\n"
result:       .ascii "The Result is \n"
##### Code Segment #####
.text
.globl main #Main function
main:
la $a0, hellomessage
li $v0, 4
syscall

la $a0, number
li $v0, 4
syscall

li $v0, 5
syscall
```

THE CODE

```
    move $a0,$v0
jal Fact
    move $s0,$v0

    la $a0,result
    li $v0,4
    syscall

    move $a0,$s0
    li $v0,1
    syscall

    li $v0,10
    syscall
    -----
Fact:
fact:
addi $sp, $sp, -8 # adjust stack for 2 items
sw $ra, 4($sp) # save return address
sw $a0, 0($sp) # save argument
slti $t0, $a0, 1 # test for n < 1
beq $t0, $zero, L1
addi $v0, $zero, 1 # if so, result is 1
addi $sp, $sp, 8 # pop 2 items from stack
jr $ra # and return
L1: addi $a0, $a0, -1 # else decrement n
jal fact # recursive call
lw $a0, 0($sp) # restore original n
lw $ra, 4($sp) # and return address
addi $sp, $sp, 8 # pop 2 items from stack
mul $v0, $a0, $v0 # multiply to get result
jr $ra # and return
```

REGISTERS VALUES

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000000
\$a0	4	0x00000078
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000001
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000078
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7fffeffc
\$fp	30	0x00000000
\$ra	31	0x00400030
pc		0x00400058
hi		0x00000000
lo		0x00000078

SAMPLE OUTPUT & INPUT

```
Hello in Factorial _Example
Enter number
5
The Result is
120
```

VALUE IN MEMORY SEGMENT

The screenshot displays a debugger interface with two main panels: 'Text Segment' and 'Data Segment'.

Text Segment: This panel shows assembly code with columns for Bkpt, Address, Code, Basic, and Source. The code includes instructions for loading addresses, adding values, and making jumps.

Bkpt	Address	Code	Basic	Source
	0x00400000	0x3c011001	lui \$1,0x00001001	17: la \$a0,hellomessage
	0x00400004	0x34240000	ori \$4,\$1,0x00000000	
	0x00400008	0x24020004	addiu \$2,\$0,0x00000...	18: li \$v0,4
	0x0040000c	0x0000000c	syscall	19: syscall
	0x00400010	0x3c011001	lui \$1,0x00001001	21: la \$a0,number
	0x00400014	0x3424001d	ori \$4,\$1,0x0000001d	
	0x00400018	0x24020004	addiu \$2,\$0,0x00000...	22: li \$v0,4
	0x0040001c	0x0000000c	syscall	23: syscall
	0x00400020	0x24020005	addiu \$2,\$0,0x00000...	25: li \$v0,5
	0x00400024	0x0000000c	syscall	26: syscall
	0x00400028	0x00022021	addu \$4,\$0,\$2	28: move \$a0,\$v0
	0x0040002c	0x0c100016	jal 0x00400058	29: jal Fact

Labels: A table on the right lists labels and their addresses.

Label	Address
(global)	
main	0x00400000
Factorial Example	
Fact	0x00400058
fact	0x00400058
L1	0x00400078
hellomessage	0x10010000
number	0x1001001d
result	0x1001002b

Data Segment: This panel shows memory values at various addresses, organized in columns for different offsets (+0, +4, +8, +c, +10, +14, +18, +1c).

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6c6c6548	0x6e69206f	0x63614620	0x69726f74	0x5f206c61	0x6d617845	0x0a656c70	0x746e4500
0x10010020	0x6e207265	0x65626d75	0x54000a72	0x52206568	0x6c757365	0x73692074	0x00000a20	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

At the bottom, there are navigation buttons and a status bar showing the current address (0x10010000 (.data)) and options for Hexadecimal Addresses, Hexadecimal Values, and ASCII.

FACTORIAL IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code is a program that calculates the factorial of a given number using recursion. Here's the breakdown of its functionality:

1. ****Data Segment****:

- Declares three strings: ``hellomessage``, ``number``, and ``result``, used for displaying messages to the user.

2. ****Code Segment****:

- ``.globl main``: Marks the beginning of the ``main`` function.
- Prints a greeting message and a prompt for the user to enter a number.
- Reads an integer input by the user using `syscall 5` (read integer).
- Calls the function ``Fact`` to calculate the factorial of the input number.
- Stores the result (factorial) in register ``$s0``.
- Prints the message "The Result is" using `syscall 4` (print string) with the address of ``result``.
- Prints the factorial stored in register ``$s0`` using `syscall 1` (print integer).
- Exits the program.

3. ****`Fact` function****:

- This function calculates the factorial of a given number recursively.
- It starts with a label ``fact`` and saves the return address and argument on the stack.
- It checks if the input number is less than 1. If true, it returns 1 as the factorial value.
- If the input number is greater than or equal to 1, it decrements the input number and makes a recursive call to itself (``fact``).
- After the recursive call returns, it multiplies the input number with the returned factorial value to calculate the factorial of the original input number.
- Finally, it returns the factorial value.

SORT IN FUNCTION CODE

Problem Statement:

-Convert The C program of the Sort. Example to Assembly Lang. By Mars

Sample Input :

I take The input numbers from the array defined in the data segment

Sample Output :

is the sorted array

CODE

```
1  #Name : Aya Mohamed Sayed Hamed
2  #Sec : 1
3  #B.NO : 9
4  #Programa 1 : Sort_Example
5
6
7
8  ##### Data Segment #####
9  .data
10 hellomessage: .ascii "Hello in Sort _Example\n"
11 spacemessage: .ascii " "
12 string:       .ascii "The array stored in the registers is \n"
13 array_print:  .ascii " 8 1 6 0 9 \n"
14 result:      .ascii "The Result after sorting is \n"
15 array: .word 8, 1, 6, 0, 9 # Example array to be sorted
16 array_size: .word 5      # Size of the array
17
18 ##### Code Segment #####
19 .text
20 .globl main #Main function
21 main:
22     la $a0, hellomessage
23     li $v0, 4
24     syscall
25
26     la $a0, string
27     li $v0, 4
```

THE CODE

NOTE:

THE SORTED FUNCTION IS TAKEN FROM THE BOOK AS YOUR INSTRUCTION

```
6  la $a0,string
7  li $v0,4
8  syscall
9
0  la $a0,array_print
1  li $v0,4
2  syscall
3      la $a0, array          # Load address of array
4      lw $a1, array_size     # Load array size
5      jal Sort
6      la $a0,result
7  li $v0,4
8  syscall
9      li $t0,0
0
1      la $t1,array
2      lw $t2,array_size
3
```

THE CODE FOR PRINT THE ARRAY AFTER SORTING

```
printarray:
    slt $t3, $t0, $t2
    beq $t3, $zero, exit_printarray
    sll $t5, $t0, 2
    add $t6, $t5, $t1
    lw $s0, 0($t6)
    move $a0, $s0
    li $v0, 1
    syscall
    la $a0, spacemessage
    li $v0, 4
    syscall

    addi $t0, $t0, 1
    j printarray

exit_printarray:
    li $v0, 10
    syscall
```

REGISTERS VALUES

Registers	Coproc 1	Coproc 0
CPU registers	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000000
\$a0	4	0x10010018
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000005
\$t1	9	0x10010070
\$t2	10	0x00000005
\$t3	11	0x00000000
\$t4	12	0x00000009
\$t5	13	0x00000010
\$t6	14	0x10010080
\$t7	15	0x00000000
\$s0	16	0x00000009
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7fffeffc
\$fp	30	0x00000000
\$ra	31	0x00400044
pc		0x004000a8
hi		0x00000000
lo		0x00000000

SAMPLE OUTPUT & INPUT

```
Hello in Sort _Example
The array stored in the registers is
 8 1 6 0 9
The Result after sorting is
0 1 6 8 9
```

VALUE IN MEMORY SEGMENT

EditExecute

Text Segment

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x3c011001	lui \$1,0x00001001	22: la \$a0,hellomessage
☐	0x00400004	0x34240000	ori \$4,\$1,0x00000000	
☐	0x00400008	0x24020004	addiu \$2,\$0,0x00000000	23: li \$v0,4
☐	0x0040000c	0x0000000c	syscall	24: syscall
☐	0x00400010	0x3c011001	lui \$1,0x00001001	26: la \$a0,string
☐	0x00400014	0x3424001b	ori \$4,\$1,0x0000001b	
☐	0x00400018	0x24020004	addiu \$2,\$0,0x00000000	27: li \$v0,4
☐	0x0040001c	0x0000000c	syscall	28: syscall
☐	0x00400020	0x3c011001	lui \$1,0x00001001	30: la \$a0,array_print
☐	0x00400024	0x34240042	ori \$4,\$1,0x00000042	
☐	0x00400028	0x24020004	addiu \$2,\$0,0x00000000	31: li \$v0,4
☐	0x0040002c	0x0000000c	syscall	32: syscall

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6c6c6548	0x6e69206f	0x726f5320	0x455f2074	0x706d6178	0x000a656c	0x54002020	0x61206568
0x10010020	0x79617272	0x6f747320	0x20646572	0x74206e69	0x72206568	0x73696765	0x73726574	0x20736920
0x10010040	0x3820000a	0x36203120	0x39203020	0x54000a20	0x52206568	0x6c757365	0x66612074	0x20726574
0x10010060	0x74726f73	0x20676e69	0x0a207369	0x00000000	0x00000000	0x00000001	0x00000006	0x00000008
0x10010080	0x00000009	0x00000005	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Labels

Label	Address
(global)	
main	0x00400000
Sort_Function	
printarray	0x00400008
exit_printarray	0x004000a0
Sort	0x004000a8
for1tst	0x004000cc
for2tst	0x004000d8
exit2	0x00400010c
exit1	0x004000114
swap	0x004000130

☒ Data☒ Text

0x10010000 (.data)

☒ Hexadecimal Addresses☒ Hexadecimal Values☐ ASCII

SORT IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code performs sorting on an array using the bubble sort algorithm. Let's break down the code step by step:

1. `**Data Segment**`:

- Declares five strings: ``hellomessage``, ``spacemessage``, ``string``, ``array_print``, and ``result``, which are used for displaying messages to the user.
- Declares an array ``array`` with initial values to be sorted.
- Declares a word ``array_size`` to store the size of the array.

2. `**Code Segment**`:

- ``globl main``: Marks the beginning of the ``main`` function.
- Prints a greeting message and a message indicating the array stored in the registers.
- Prints the content of the array using `syscall 4`.
- Calls the ``Sort`` function to sort the array.
- Prints a message indicating the sorted array.
- Prints the sorted array using `syscall 1`.
- Exits the program using `syscall 10`.

3. `**`Sort` function**`:

- Begins with a label ``Sort``.
- Allocates space on the stack for 5 registers.
- Saves the current register values onto the stack.
- Initializes registers ``$s0``, ``$s1``, ``$s2``, ``$s3``, and ``$ra`` with values passed as arguments (``$a0`` and ``$a1``).
- Starts a loop labeled ``for1st`` to iterate over each element of the array.
- Inside the loop, another loop labeled ``for2st`` compares adjacent elements and swaps them if they are in the wrong order.
- The ``swap`` function is called to perform the swapping operation.
- Once the outer loop completes, the sorted array is returned.

4. `**`swap` function**`:

- Swaps the values of two elements in the array.